

Organização e Arquitetura de Computadores

Aritmética Binária

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Bases Numéricas

- Número é um conceito abstrato que utilizamos para realizar contagens
 - Um número representa uma certa quantidade
- Para representar essas quantidades numéricas, utilizamos um conjunto de símbolos distintos
- A quantidade de símbolos delimita quantos valores distintos representamos usando apenas um caractere

A Base Decimal

- Nosso sistema numérico possui dez símbolos distintos, e portanto o classificamos como decimal

0 1 2 3 4 5 6 7 8 9

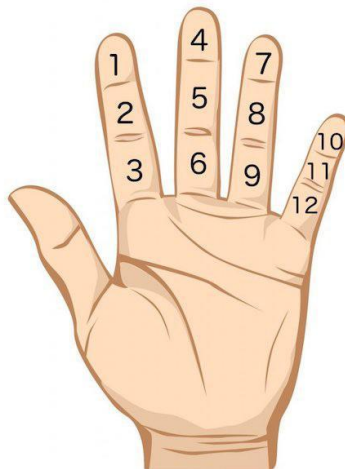


dez símbolos

- Por que decimal?

Por que decimal?

- Nem sempre utilizou-se a base decimal
 - Egípcios utilizavam base 10
 - Sumérios e babilônicos utilizavam base 60
 - Algumas civilizações utilizavam base 12
 - Algumas só contavam até 3...
- Possivelmente, a base decimal surgiu pela quantidade de dedos nas mãos
- Talvez tenha sido a primeira base a receber símbolos aritméticos e ser difundida



A base 2 - Binário

- Apesar de estarmos vinculados a base 10, circuitos lógicos operam apenas em dois estados
 - Ligado ou desligado (há corrente ou não há corrente)
- Portanto, computadores utilizam aritmética em outra base numérica
- Surge a necessidade de entendermos a base binária!

0 1



dois símbolos

Por que?

- Não é comum termos que lidar com bases binárias no dia-a-dia (mesmo desenvolvedores)
 - Porém, as vezes é necessário entendermos essas estruturas quando estamos em baixo nível de abstração (assembly, CPU)

Notação Posicional

- Em um sistema numérico em notação posicional:
 - O dígito simboliza o valor
 - A posição do dígito simboliza a potência relacionada a esse valor (com relação a sua base)
- Na base decimal:

Diagram illustrating the positional notation for the decimal number 1732_{10} . The number is expanded into its positional components:

$$1 \times 10^3 + 7 \times 10^2 + 3 \times 10^1 + 2 \times 10^0$$

Notação Posicional

- Como essa notação vale para toda base numérica, o mesmo é válido para a base binária
 - A única diferença é que as potências tem sempre base 2

$$11010_2$$
$$1x2^4 + 1x2^3 + 0x2^2 + 1x2^1 + 0x2^0$$

Generalizando para qualquer base

- Podemos generalizar essa ideia para qualquer base numérica desejada
- É importante lembrar que precisamos de símbolos suficientes para representar uma base
- Para bases maiores que 10, precisaremos de símbolos além daqueles que já conhecemos como os numéricos...

Do decimal ao binário

Decimal	Binário
0	0
1	1
3	11
6	110
15	1111
18	10010

Conversão binário-decimal

- Acabamos de ver como fazer
- Basta utilizar a regra de numeração posicional

10_2



$1 \times 2^1 + 0 \times 2^0$

1101_2



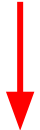
$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

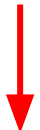
Conversão decimal-binário

- Há duas maneiras de fazer essa conversão
- A primeira envolve acumular potências de dois (menores que o valor) até representar o número

53_{10} 

$$32 + 16 + 4 + 1$$

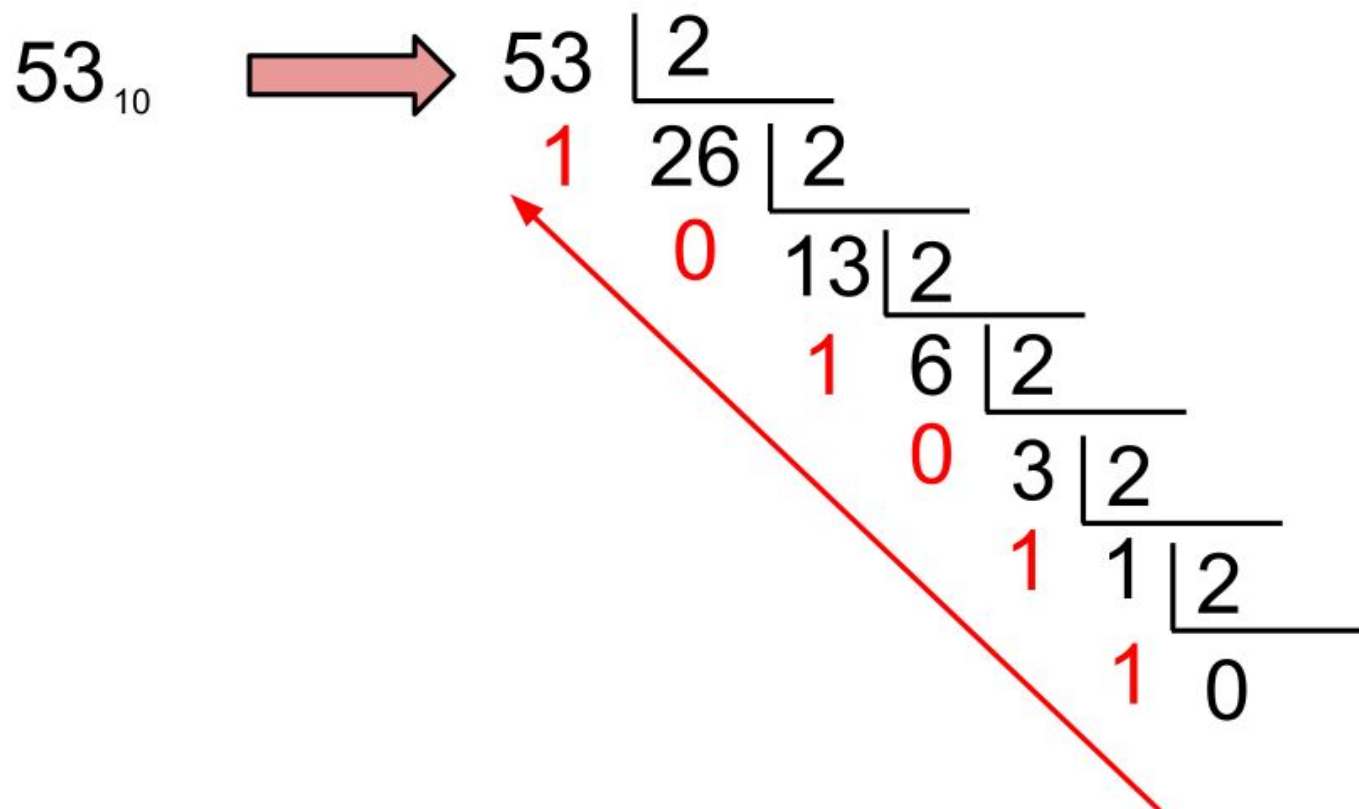

$$2^5 + 2^4 + 2^2 + 2^0$$



110101

Conversão decimal-binário

- A segunda envolve dividir o número por 2 enquanto acumula os restos



Bit e Byte

- Utilizamos nomes especiais para sinalizar valores binários com certos tamanhos
 - **bit**: valor binário com apenas um algarismo (0 ou 1)
 - **byte**: valor binário com oito algarismos (entre 0 e 255)
- A memória do seu computador só opera em **bytes**
- Por que sua internet oferece uma velocidade de download, mas o que você registra é diferente?

Bit

0

1

Byte

00000100

00110111

00000001

11111111

Prefixos - kilo, mega, giga, ...

- A quantidade de bytes que utilizamos é difícil de ser representada em unidades simples
- Usamos prefixos para denotar multiplicidades de bits e bytes
- Esses prefixos vem em dois tipos:
 - Baseado em potências de 10
 - Baseado em potências de 2

Prefixos - kilo, mega, giga, ...

- Potência de 10
 - kilobyte (kB) - 1000 bytes
 - megabyte (MB) - 1000^2 bytes
 - gigabyte (GB) - 1000^3 bytes
 - terabyte (TB) - 1000^4 bytes
 - ...
- Potências de 2
 - kibibyte (KiB) - 2^{10} bytes = 1024 bytes
 - mebibyte (MiB) - 1024^2 bytes
 - gibibyte (GiB) - 1024^3 bytes
 - tebibyte (TiB) - 1024^4 bytes
 - ...
- Por que um disco de 1 Terabyte contém apenas aproximadamente 931 GB?

A base 16 - Hexadecimal

- Muitas vezes, representar bytes é cansativo e ocupa muito espaço
- Nem sempre os valores são grande a ponto de representarmos com prefixos
- Utilizamos uma base numérica maior (hexadecimal)
- Precisaremos emprestar símbolos do alfabeto, visto que só temos dez símbolos numéricos na base 10

0 1 2 3 4 5 6 7 8 9 a b c d e f



dezesseis símbolos

Conversão hexadecimal - decimal

- Basta utilizar a notação posicional

$$0x10af_{16} \rightarrow 1 \times 16^3 + 0 \times 16^2 + 10 \times 16^1 + 15 \times 16^0$$

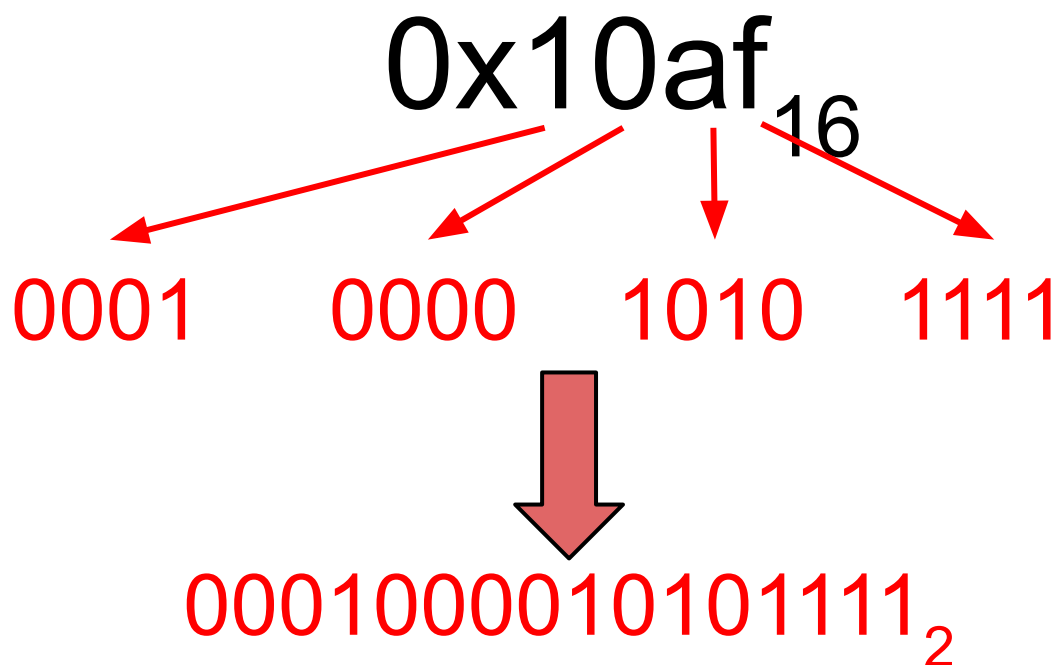
ATENÇÃO: costumamos preceder valores hexadecimais pelo prefixo **0x**

Conversão hexadecimal - decimal

- Há duas maneiras de realizar essa conversão
 - Realizar a conversão em dois passos
 - hexadecimal -> decimal -> binário
 - Realizar a conversão direta

Conversão hexadecimal - decimal **direta**

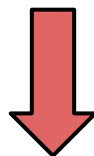
- Considere a seguinte ideia:
 - Um algarismo hexadecimal armazena valores de 0 a 15
 - Isso corresponde a 4 bits (meio byte)
 - Basta representar cada algarismo hexadecimal como um binário de 4 bits



Conversão binário - hexadecimal

- Basta inverter o processo da conversão hexadecimal -> binário
 - A cada quatro algarismos, corresponde um hexadecimal

10110011111001₂



10 1100 1111 1001₂



2



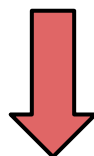
12



15



9



0x2cf9

Exercícios

1. Realize as conversões entre bases:
 - a. 01100111_2 (bin -> dec)
 - b. 11001011010000111_2 (bin -> hex)
 - c. 82_{10} (dec -> bin)
 - d. $0xaf60_{16}$ (hex -> dec)
 - e. $0xab10f_{16}$ (hex -> bin)
 - f. 912342_{10} (dec -> hex)
 - g. 131_{10} (dec -> bin)
2. Represente os seguintes valores com prefixos:
 - a. 1073741824 bytes (em GiB)
 - b. 1.000.000.000 bytes (em GB)
 - c. 132 KB (em bytes)
 - d. 12 GB (em GiB)
 - e. 132 MB (em MiB)
 - f. 11512 Mb (em GB)

Exercícios

3. Considerando que um inteiro (sem sinal) em um computador pessoal armazena valores de até 4 bytes, qual o valor máximo que podemos armazenar em um inteiro?
4. Considere o valor abaixo:

1111001011110111101111010101000010000₂

Quantos bytes há nesse intervalo de bits?

Soma - Em Decimal e Binário

- O algoritmo para realização da soma decimal é ensinado desde cedo

$$\begin{array}{r} 115 \\ + 47 \\ \hline 162 \end{array}$$

Soma - Em Decimal e Binário

- Assim como muito do que vimos anteriormente, podemos estender essa ideia para outras bases, como a base binária
- Na base binária, o cálculo é muito mais simples
- Estamos limitados a dois símbolos, e portanto so há quatro possibilidades
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 0 \dots$ e “vai” 1



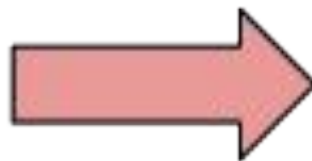
10_2

Soma - Em binário

$$\begin{array}{r} 1000\overset{1}{1}1011 \\ + 11011000 \\ \hline 111110011 \end{array}$$

Soma - Em Decimal e Binário

$$\begin{array}{r} \overset{1}{15} \\ + 7 \\ \hline 22 \end{array}$$



$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1111 \\ + 111 \\ \hline 10110 \end{array}$$

Subtração em Binário

- Assim como na soma, vamos replicar o mesmo algoritmo adaptado para a base 2
 - $0 - 0 = 0$
 - $1 - 0 = 1$
 - $1 - 1 = 0$
 - $0 - 1 = 1 \dots$ e “empresta” $10_2 = 2_{10}$

Subtração em Binário

$$\begin{array}{r} 10\overset{0}{\cancel{1}}0011 \\ - 11001 \\ \hline 1010 \end{array}$$

Subtração em Binário

$$\begin{array}{r} \textcolor{red}{0} \textcolor{red}{2} \textcolor{red}{0} \textcolor{red}{2} \\ \cancel{1} \cancel{0} \cancel{1} 0 0 1 1 \\ - \quad \quad 1 1 0 0 1 \\ \hline \quad \quad 1 0 1 0 \end{array}$$

Subtração em Binário

$$\begin{array}{r} \text{0} \text{ 1} \text{ 2} \\ \text{0} \text{ 2} \text{ 0} \text{ 2} \\ \hline 1010011 \\ - \quad 11001 \\ \hline 0111010 \end{array}$$

Subtração - Decimal e Binário

$$\begin{array}{r} 15 \\ - 6 \\ \hline 9 \end{array}$$



$$\begin{array}{r} 1111 \\ - 110 \\ \hline 1001 \end{array}$$

Multiplicação Binária

- Sem nenhuma surpresa, o mesmo padrão dos exemplos anteriores
- Em base 10, o algoritmo de multiplicação necessita da soma como passo intermediário
 - Em base 2, isso também ocorre
 - $0 \times 0 = 0$
 - $1 \times 0 = 0$
 - $0 \times 1 = 0$
 - $1 \times 1 = 1$

Multiplicação Binária

$$\begin{array}{r} 1011 \\ \times 110 \\ \hline 0000 \\ + 1011 \\ 1011 \\ \hline 1000010 \end{array}$$

Divisão Binária

- O que vocês acham?
 - Idêntico às operações base 10
- **Desafio: tentem deduzir sozinhos em casa**

Representação de valores sinalizados

- Até o momento, observamos operações com valores naturais
- E se precisarmos representar um número inteiro?
- Poderíamos utilizar a operação de subtração, mas isso é muito trabalhoso...
- Precisamos encontrar uma maneira de representar o sinal (positivo ou negativo)
 - Lembre-se: o computador só sabe armazenar bits
- Observação: vamos trabalhar com o escopo de byte de espaço em todos os exemplos

Representação sinal-magnitude

- Sacrificamos o bit mais significativo para representar o sinal do número
 - LSB (least significant bit) - menos significativo/unidade
 - MSB (most significant bit) - mais significativo
- MSB = 0 -> positivo
- MSB = 1 -> negativo

Representação Sinal-Magnitude



Representação sinal-magnitude

- Vantagem: simples
- Desvantagens
 - Há duas maneiras de representar o zero:
 - 10000000_2 e 00000000_2
 - Adição e subtração necessitam de passos extras para analisar o sinal
 - Comparação entre valores requer passos extras para analisar o sinal

O conceito de complemento

- Dado um número binário qualquer n , seu complemento é dado por:

$$\bar{n} = k - n$$

- onde k é o maior algarismo na base 2
- para o contexto de um byte, o maior algarismo é 11111111_2

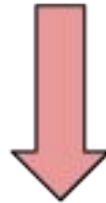
$$\bar{n} = 11111111_2 - n$$

- Em binário, essa operação é equivalente a inverter os bits do número original
- **Importante:** a operação do complemento só faz sentido se limitarmos o tamanho do valor (em bits/bytes)

Complemento a 1

- Utilizaremos o complemento de um número para representar esse número com valor negativo

n 00001100 (12)



$\bar{n}$ 11110011 (-12)

- repare: basta inverter todos os bits!

Complemento a 1

- Agora ficou muito mais simples somar dois números inteiros: basta usar a soma binária comum
 - é preciso considerar o bit extra como *carry back* e somá-lo na unidade

$$\begin{array}{r} 16 \\ - 7 \\ \hline 9 \end{array} \quad \longrightarrow \quad \begin{array}{r} \overset{1}{0} \overset{1}{0} \overset{1}{0} 10000 \quad (16) \\ + 11111000 \quad (-7) \\ \hline \overset{1}{1} 00001000 \end{array}$$

Complemento a 1

- Agora ficou muito mais simples somar dois números inteiros: basta usar a soma binária comum
 - é preciso considerar o bit extra como *carry back* e somá-lo na unidade

$$\begin{array}{r} 16 \\ - 7 \\ \hline 9 \end{array} \quad \longrightarrow \quad \begin{array}{r} \overset{1}{0} \overset{1}{0} \overset{1}{0} 10000 \quad (16) \\ + 11111000 \quad (-7) \\ \hline \overset{1}{1} 00001000 \end{array}$$

carry back

Complemento a 1

- Agora ficou muito mais simples somar dois números inteiros: basta usar a soma binária comum
 - é preciso considerar o bit extra como *carry back* e somá-lo na unidade

$$\begin{array}{r} 16 \\ - 7 \\ \hline 9 \end{array} \quad \longrightarrow \quad \begin{array}{r} \overset{1}{} \overset{1}{} \overset{1}{} \\ 00010000 \quad (16) \\ + 11111000 \quad (-7) \\ \hline 00001000 \\ + \overset{1}{} \text{ carry back} \\ \hline 00001001 \end{array}$$

Complemento a 1

- Vantagens
 - Operações aritméticas são mais simples: basta somar ambos os números considerando o *carry back*
 - Comparações são simples: basta inverter os números e somá-los
- Desvantagens
 - Ainda há duas representações para zero
 - 00000000_2 e 11111111_2
 - Considerar o *carry back* é um tanto chato

Complemento a dois

- Para solucionar os problemas anteriores, surge o complemento a 2
- Dado um número binário n , seu complemento a 2 n_{c2} é dado por:

$$n_{c2} = \bar{n} + 1$$

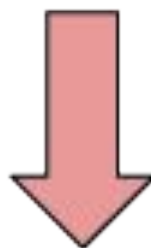
- ou, somar 1 no complemento a 1

Obtendo o valor do complemento a dois

- Há duas maneiras:
 - **Método 1**
 - Gerar o complemento a 1 do número
 - Somar 1 no valor do complemento
 - **Método 2**
 - Partindo do LSB até o MSB, procurar o primeiro bit 1 do número
 - Inverter todos os bits **após** o bit encontrado

Obtendo um valor em complemento a dois

00110100 (52)



11001100 (-52)

Somando em complemento a dois

- Somar dois números em complemento a dois é muito simples
 - Basta realizar a soma normalmente!
 - Só é necessário considerar o *overflow*
 - Quando somamos, é possível que o resultado ultrapasse os limites da nossa quantidade de bits
 - Se a soma de dois números positivos é negativa, há *overflow*
 - Se a soma de dois números negativos é positiva, há *overflow*
 - Caso contrário, não há *overflow*

Overflow

- $00110111_2 + 01000011_2 = 01111010_2$
- $10001111_2 + 01111111_2 = 00001110_2$
- $11111111_2 + 11111111_2 = 11111110_2$
- $00111111_2 + 01100000_2 = 10011111_2$ **OVERFLOW**

Complemento a dois

- Vantagens
 - Só há uma representação para zero: 00000000_2
 - Operações aritméticas são muito simples
 - Comparações são igualmente simples
- Desvantagens
 - Overflow ?

Exercícios

- Calcule o resultado das seguintes operações:
 - $11011101_2 + 11010010_2$
 - $10110101_2 - 100110_2$
 - $1011111_2 \times 11_2$
- Represente os seguintes valores em complemento a dois
 - 11011000_2
 - 00011101_2
 - 11011110_2
- Calcule as seguintes operações binárias em complemento a um e sinalize o resultado em decimal:
 - $4_{10} + 18_{10}$
 - $118_{10} - 14_{10}$
 - $-58_{10} + 49_{10}$
 - $102_{10} - 12_{10}$

Exercícios

- Dadas as somas em complemento a dois abaixo, sinalize o seu resultado (em binário). Caso haja overflow, escreva **OVERFLOW**
 - $01101101_2 + 10011010_2$
 - $11101101_2 + 10011010_2$
 - $01111101_2 + 00011010_2$
 - $00000001_2 + 11111111_2$
- Calcule as seguintes operações binárias em complemento a 2 e sinalize o resultado em decimal:
 - $18_{10} - 47_{10}$
 - $120_{10} + 3_{10}$
 - $100_{10} - 53_{10}$
 - $-90_{10} + 18_{10}$